

Scoring, Not Solving

A verification oracle for materials physics

*It is harder to solve a problem than to verify a solution —
so verification is what has been built.*

Application: durable ultra-wide-bandgap devices for harsh operating conditions.

Javier Flores · 2026-06-25

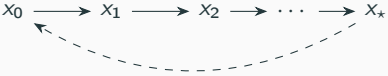
The asymmetry

The asymmetry

- The defining property of **NP**: a *certificate* (a witness) is checkable in polynomial time even when finding it is exponentially hard. Sudoku, factoring, satisfiability — hard to solve, trivial to check.
- /physics lives on the **cheap side of that gap on purpose**: it never searches for the state; it grades a state it is handed.
- The whole system is engineered to make *checking* fast, exhaustive, and trustworthy.

A pure oracle

Solve $x_0 \longrightarrow x_1 \longrightarrow x_2 \longrightarrow \dots \longrightarrow x_*$ iterate until $F(x)=0$ — **expensive**



Score $x \xrightarrow{\text{score}} (\|F(x)\|, \text{cert})$ one pass — **cheap**

- /physics is a pure oracle: it owns no training loop and no sample-selection logic (`arch-01-purpose` §72–75).
- The scoring principle holds at every length-and-time scale — femtoseconds to years, atom to device (`arch-21-multiscale-state` §87–88).

The cost ladder

Tier	Budget	Cadence	Shape of the work
T0	$\leq 10 \mu\text{s}$, closed-form	every gradient step	a few arithmetic ops
T1	$\leq 10 \text{ ms}$, small lin. alg. / 1-D quad.	per batch (sampled)	a small dense solve
T2	$\leq 10 \text{ s}$, bounded integral on a grid	per epoch (cached)	a sweep over a fixed mesh
T3	$\leq 10 \text{ min}$, iterative solve / PDE	calibration only	a fixed-point loop to convergence

The corresponding *solve* — a full iterative loop from scratch — is the **T3** rung, minutes. The oracle keeps scoring on the **T0/T1** rungs, “fast by construction” ([arch-07-pipeline §28–32](#)).

The certificate

- “an inert s-expression carrying scalar verdicts plus numeric witnesses for failures” (`arch-12-cert` §19–22) — an NP-style certificate: cheap to check, carries its own evidence.
- A failure carries a **witness** — the exact offending input, e.g. the pair of values that broke a consistency rule (§109–119).
- Ten obligations: symmetry, bounds, analytic limits, reference battery, conservation, consistency, bulk–boundary, versioning, surrogate validity, adjoint existence (§24–40).

Passed



Pending



Failed

meet $\sqcap = \min$
any Failed
 $\Rightarrow$ Failed

Verdicts combine by a semilattice meet

(`impl-07` §56–60).

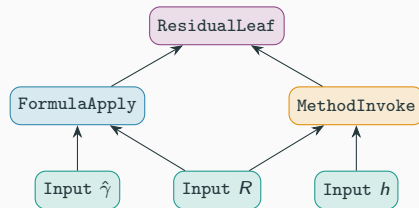
Physics as a language

The alphabet and grammar

- 12 computational **methods** (+3 sub)
- 20 abstract-property **templates**
- 125 named **formulas** (+2 markers)
- 11 observable **bundles**
- 19 **residual categories**
- 10 **cert obligations**, 4 **typeclasses**
- “instances are programs in this vocabulary” (`arch-09-vocabularies §9.1`).
- A finite, decidable grammar makes the set of well-formed questions **enumerable and exhaustive** — “there should be no more questions one can ask.”

The abstract syntax tree

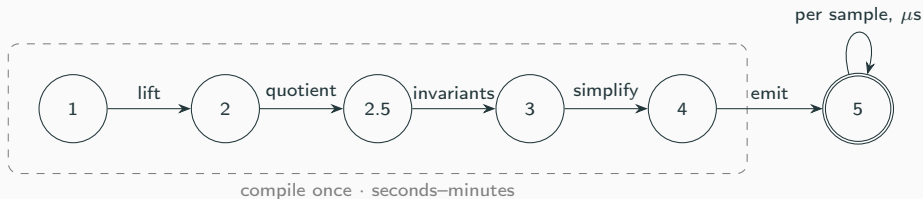
- The PhysicsGraph: “a typed, hash-consed, content-addressed directed acyclic graph” (`arch-06-physics-graph`).
- Three node kinds — the grammar’s terminals: Input, FormulaApply, MethodInvoke (§6.2).
- The 125 formulas and 12 methods are **typing rules** for these nodes (§6.5).
- Composing observables *is* composing subgraphs — a new observable is a new sentence, not new code.



Shared child *R* feeds two parents: hash-consing makes it a DAG, not a tree.

The compiler

/physics is a compiler



Stage	Compiler analogue	What happens
1 Symbolic lift	parse / front-end	instantiate templates → typed graph
2 Symmetry quotient	optimization pass	rewrite collapsing equal work — up to 48× smaller
2.5 Invariant synthesis	semantic analysis	a reduced basis fixed by the symmetry group
3 Algebraic simplify	classic IR optimization	hash-consing, cross-formula CSE, tearing
4 Lowering + adjoint	codegen + reg. alloc.	compression plans, synthesize adjoints, emit
5 Runtime kernel	the executable	straight-line numeric code

Staging: pay the symbolic cost once



- The compile/runtime split is **multi-stage programming / partial evaluation**: all symbolic reasoning, symmetry reduction, and optimization is resolved at compile time ([arch-07-pipeline §7.6](#)).
- This is *why* checking is cheap: the system residualizes the physics once, then evaluates the residual forever.

Structure and guarantees

A type system over the quantities

- Four Layer-0 typeclasses (`arch-10-typeclasses`):
Quantity (value), Sampleable (shape),
HasAnalyticStructure (law), DiscreteStructure
(invariant).
- Cert checkers are “generic functions over the
typeclasses” (§45–50) — one checker per axis, reused
across all physics.
- The same axes carry the algebra that makes
composition lawful: an obligation-7
DiscreteStructure morphism; SymbolicTensorOps
as a colored operad (`arch-20` §98–103); the Reynolds
projector $P = \frac{1}{|G|} \sum_g \rho(g)$ onto the symmetry-fixed
subspace, which provably preserves
positive-semidefiniteness (`arch-19`).

$$\begin{array}{ccc} V & \xrightarrow{T} & W \\ \rho(g) \downarrow & & \downarrow \sigma(g) \\ V & \xrightarrow{T} & W \end{array}$$

T equivariant $\Leftrightarrow$ the square commutes.

`Address[D] = Hash(domain_sep, schema_version, canonical_bytes)` ([arch-20-representations §20.4](#))

- **Memoization & dedup** — identical sub-expressions collapse to one node (hash-consing).
- **Tamper-evidence** — changing any payload changes its hash and trips the freeze comparison ([arch-12-cert §155–160](#)).
- **Structural sharing** — the whole IR is a Merkle DAG, like Git commits or a build cache.

Differentiation as a first-class output

- Stage 5 emits **gradients** $\partial R / \partial \hat{x}$ alongside the residuals — reverse-mode AD built into the kernel.
- The **implicit-function-theorem adjoint** makes the gradient of a fixed-point solve cost “one extra linear solve, independent of forward iteration count” (`arch-07-pipeline` §133–137) — the verify-cheap asymmetry, now for derivatives.
- The bridge to the learned model is a typed **calling convention**: the oracle returns $\partial R / \partial \hat{x}$; the consumer “brings its own backward” to chain $\partial \hat{x} / \partial \theta$ (`arch-16-pino-bridge`). Three exports — `Generate` / `Validate` / `Import` — form the interface.

Well-formedness, checked by the compiler

- Registration is a **decidable gate**: every applicability predicate is “first-order decidable — finite case analysis on typeclass tags, not numeric thresholds” (`impl-04-formulas`).
- The **adjoint gate** fails the build when a derived gradient disagrees with a finite-difference check beyond τ_{adj} (`impl-07 §7.5`).
- Compose-time **refusals** reject inconsistent compositions by tag comparison — an unprovenanced coefficient, or a double-counted temperature path (`arch-12 §12.0.3`).
- The 13-phase build sequence (`impl-10-build-sequence`) is the state machine that enforces all of it.

Where each guarantee comes from

Guarantee	Comes from
Speed	staged compilation + the verify < solve asymmetry
Exhaustiveness	a closed grammar + decidable applicability
Trust	checkable certificates + content-addressed tamper-evidence
Composability	an AST with a type system
Gradients	first-class AD + implicit-function adjoints

The physics is the domain; the machinery above is what makes it a system.

Held in reserve for domain-expert questions:

- The full closed-vocabulary tables (12 methods, 20 templates, 125 formulas, 11 bundles).
- The 19 residual categories (`arch-11-residuals` §11.1): 9 equation-of-motion, 3 structural, 5 algebraic-identity, 2 constraint.
- The 10 cert obligations in full (`arch-12-cert` §24–40).
- The tier/cost/cadence table with per-tier examples (`impl-07` §175–191).
- The error model: a per-residual budget composed from input σ , model-form error, compression truncation, dressing staleness, and coefficient-provenance σ (`arch-11-residuals` §11.7).

Appendix — concept to where it lives

Concept	In /physics	Source
NP certificate / verify < solve	the cert + the scoring principle	arch-12, arch-01, arch-21
Complexity tiers of checking	cadence T0–T3	impl-07 §175–191
Formal language / closed grammar	the vocabularies	arch-09
AST + typing rules	PhysicsGraph, 3 node kinds	arch-06
Compiler passes (CSE, lowering)	Stages 1–5	arch-07
Partial evaluation / staging	compile-once / run-per-sample	arch-07 §7.6
Type system / typeclass dispatch	4 Layer-0 typeclasses	arch-10, arch-12
Category theory (operad, morphism)	SymbolicTensorOps, obligation 7	arch-20, arch-19
Lattice theory	verdict semilattice meet	impl-07 §56–60
Merkle DAG / content addressing	Address, hash-consing	arch-20 §20.4
Reverse-mode AD / implicit adjoint	gradient exports	arch-07 §133–137, arch-16
Decidability / build automaton	applicability gate, refusals, 13-phase	impl-04, impl-10, arch-12